



Query Optimization — Chapter 16

2025年12月

School of Computer Science



北京邮电大学

16.1 Overview

- **Motivation:**

- A query statement corresponds to **several** equivalent expressions
 - Different costs** for equivalent evaluation **plans**

- **Query optimization**

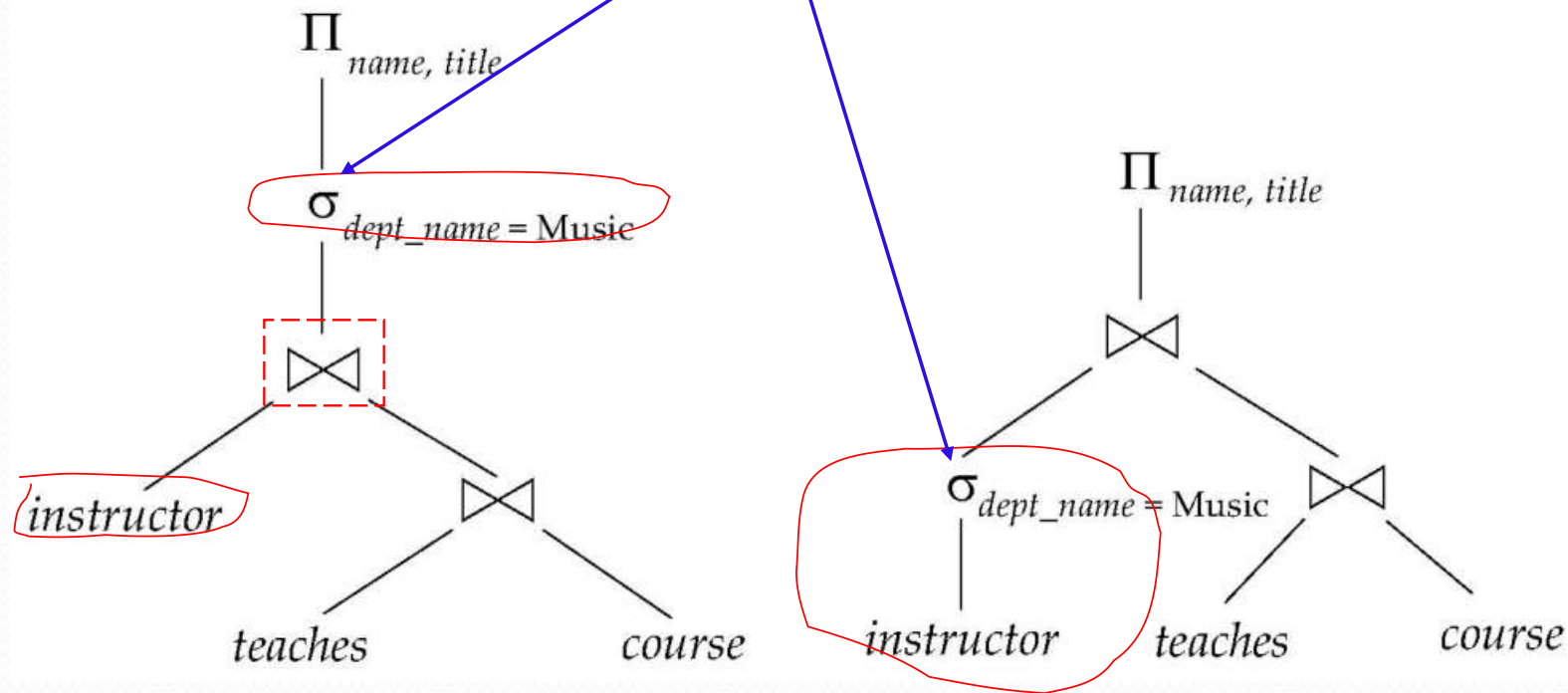
- needed to choose an **efficient** evaluation plan with **lower** costs

- Example:

```
select *  
  from r, s  
 where r.A = s.A
```

Overview

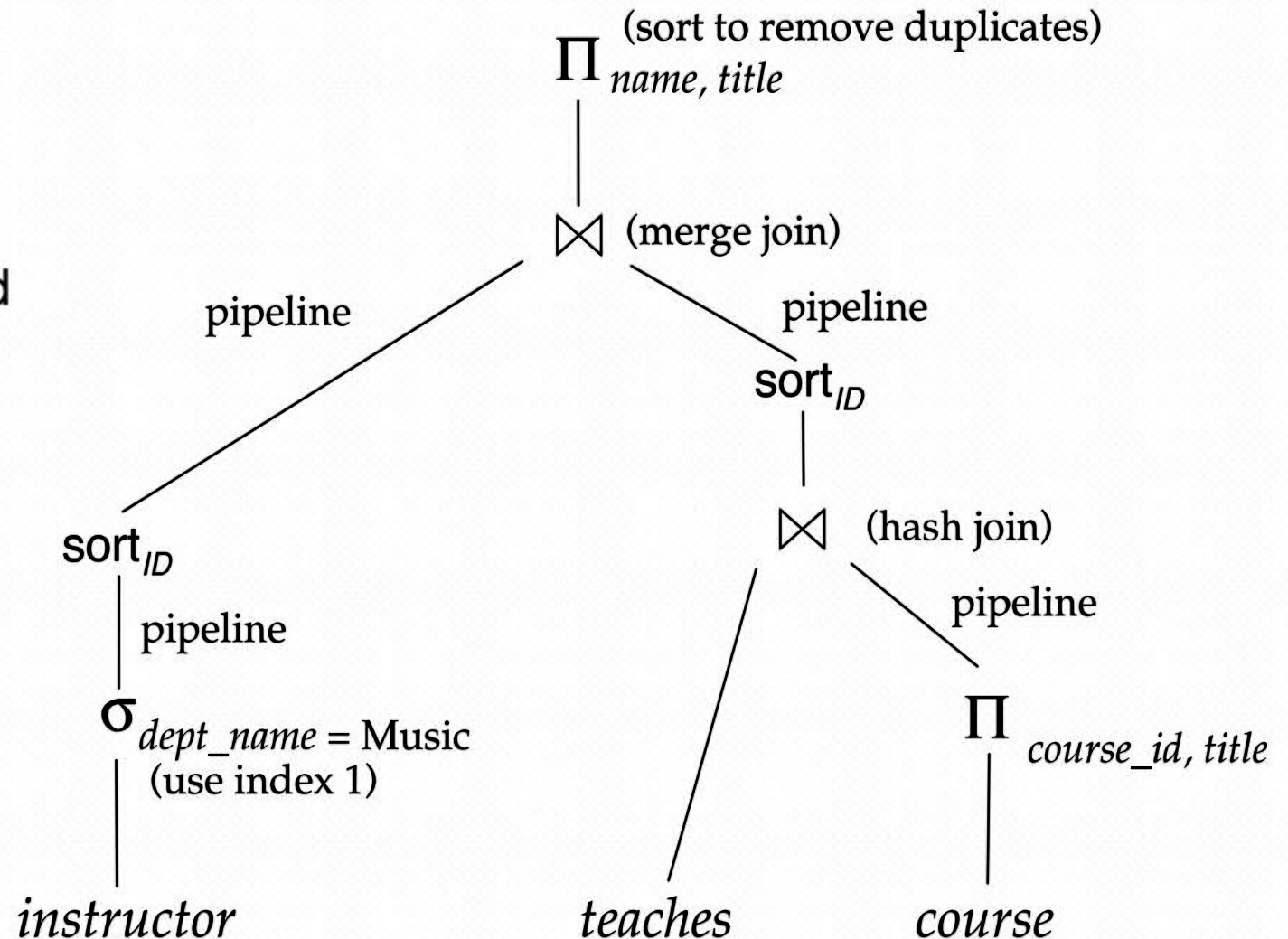
- **Example:** Find the *names* of *instructors* in department *Music* together with the *title* of all *courses* that instructors *teach*
- **Alternative ways:** evaluating a given query
 - equivalent expressions, but *with different query cost*



Overview

- **Def: Evaluation plan**
- ▣ **what algorithm** is used for each operation
- ▣ **how execution** of the operations is coordinated

An evaluation plan



Query Optimization Procedure

- Step 1. **Generating** the **equivalent** query trees/expressions, by **transforming** of relational algebra expressions using **equivalence rules**
- Step 2. **Generating** the alternative **evaluation plans**, by annotating the resultant equivalent expressions with algorithms for each operations 
- Step 3. **Choosing** the optimal or near-optimal plan based on the **estimated cost**, by
 - ▣ **heuristic optimization**
 - ▣ **cost-based optimization**

16.2 Transformation of Relational Expressions

■ **Def: Equivalent Expression**

- Two **relational algebra** expressions **are equivalent**, if **two expressions** generate the **same** set of tuples on all legal database instances,
- **The equivalent rules** transform relational expression in query optimization

Equivalence Rules

- **Rule 1.** (选择串接律, 将1个选择操作分解为2个选择操作)

Conjunctive selection operations can be deconstructed into a **sequence** of **individual** selections

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

- **Rule 2.** (选择交换律)

Selection operations are commutative :

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Equivalence Rules

■ Rule 3. (投影串接律)

Only the **final** operation in a sequence of project operations is needed

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

□ note: it is required that $L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$

□ e.g. $\Pi_{\{ID\}} \{ \Pi_{\{ID, name\}}(instructor) \}$
 $= \Pi_{\{ID\}}(instructor)$

Equivalence Rules

■ Rule 4. (连接操作代替选择和笛卡尔乘积)

Selections can be combined with Cartesian products and theta joins

a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$

□ note: definition of θ join

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

□ costs of the right-hand are less than that of the left-hand

□ $\sigma_{dept-name="Music"}(instructor \bowtie_{instructor.ID=teaches.ID} teaches)$
 $= instructor \bowtie_{(dept-name="Music) \wedge (instructor.ID=teaches.ID)} teaches$

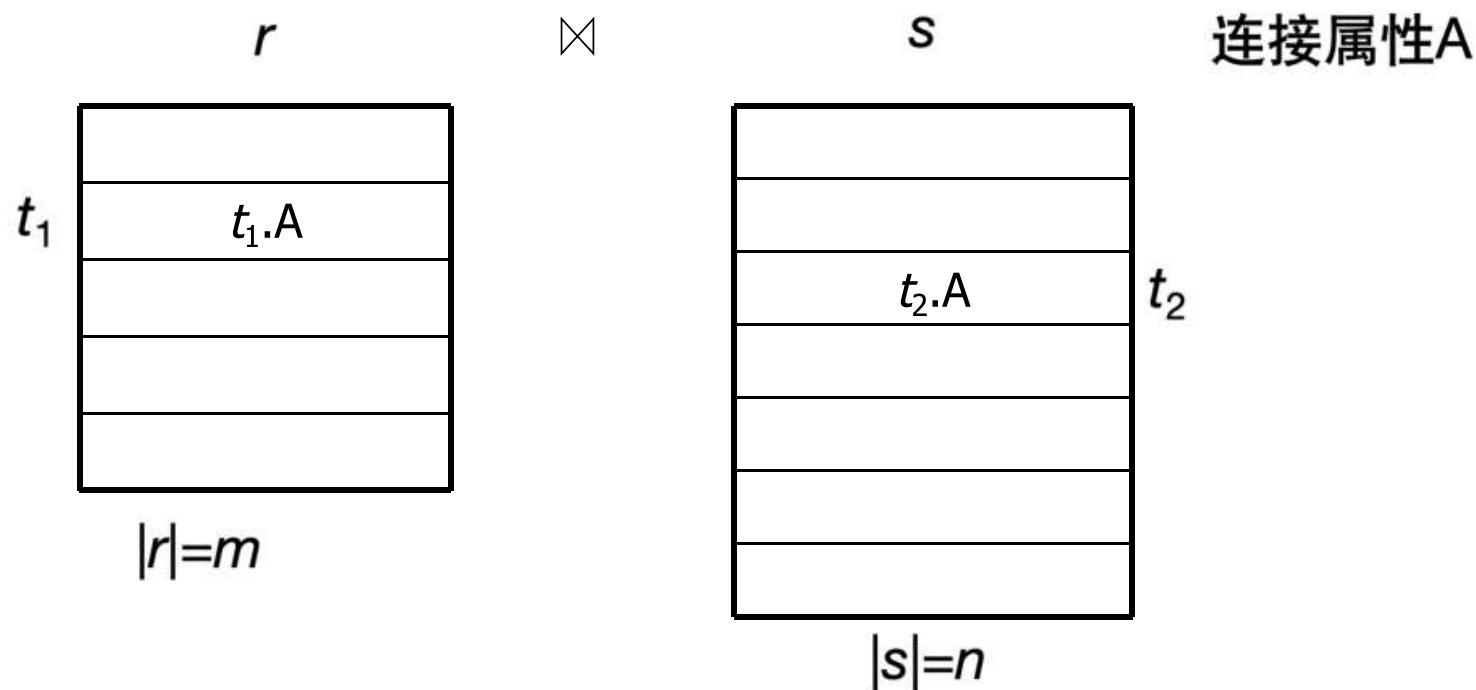
Equivalence Rules

■ Rule 5 (连接操作可交换)

Theta join operations are *commutative*

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

- the expression with smaller size should be arranged as the left one in join operation
- 连接操作中, relation of smaller size作为outer relation, 以降低 the cost of join operation



Nested-loop join原理： 针对 r 中元组 t_1 ， 检查 s 中的元组 $t_2.A$ ， $t_1.A=t_2.A$ ？

方法：

For r 中每个元组 t_1 **/*扫描**

按照 $t_1.A$ ， **查找** s 中满足 $t_1.A=t_2.A$ 的元组 t_2 ，

合并元组 t_1 和 t_2

Equivalence Rules

- 假设:

1. $|r|=m, |s|=n$

2. $m < n, \quad n/m = k > 1$

- 给定 r 中的元组 t_1 , 采用B/B+树索引、二分搜索机制, 根据 $t_1.A$, 查找 s 中满足 $t_1.A=t_2.A$ 的元组 t_2 , 树的degree为 d , 所需cost正比于树高, 为: $O(\log_d n)$

$r \bowtie s$ 的cost为: $O(m + m \log_d n)$

$s \bowtie r$ 的cost为: $O(n + n \log_d m)$

- 如果没有索引:

$O(m + m * n)$ **vs** $O(n + n * m)$

$O(m + m * n) = O(m * n) = O(n + n * m)$

$m + m \lg n$ vs $n + n \lg m$

- $m < n$, $n/m = k > 1$, $n = k * m$
- $n \lg m - m \lg n$
 - $= k * m \lg m - m (\lg k * m)$
 - $= k * m \lg m - m \lg k - m \lg m$
 - $= (k-1) m \lg m - m \lg k$
- 一般情况下, $(k-1) m \lg m - m \lg k > 0$
 - e.g. $|r|=m=500$, $|s|=n=1000$, $k = n/m = 2$
 $(k-1) m \lg m = 500 \lg 500 > m \lg k = 500 \lg 2$
- **结论:** r 作为外关系, 连接操作成本较低

Equivalence Rules

- **Rule 6.** (连接操作的结合率, associative)

- a. natural join operations are associative

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- b. *Theta joins* are associative:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3),$$

where θ_2 involves attributes from **only** E_2 and E_3

Equivalence Rules

- **Rule 7.** Selection operation distributes over theta-join
(选择操作对于连接操作的分配率,选择条件下移)
 - a. attributes in θ_0 involve **only** the attributes of one of the expressions ,
e.g. E_1 , being joined

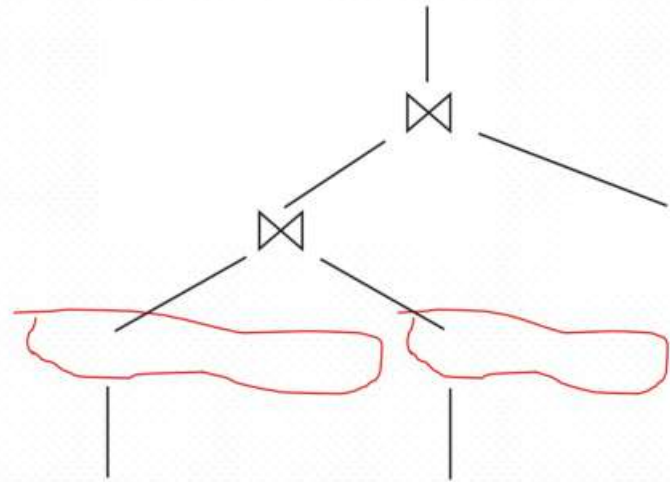
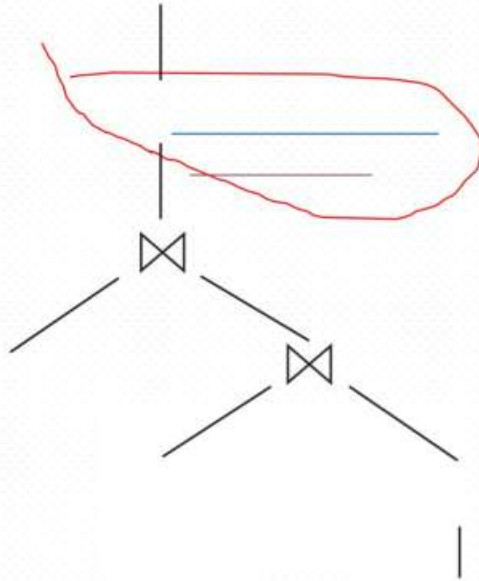
$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- b. attributes in θ_1 involves only attributes of E_1 ,
attributes in θ_2 involves only attributes of E_2 ,

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

Example: Rule 7

- **Note:** When **Rule 7** is applied, the **size** of relations participating in **theta-join** operation is **reduced**, thus evaluation costs are reduced



Equivalence Rules

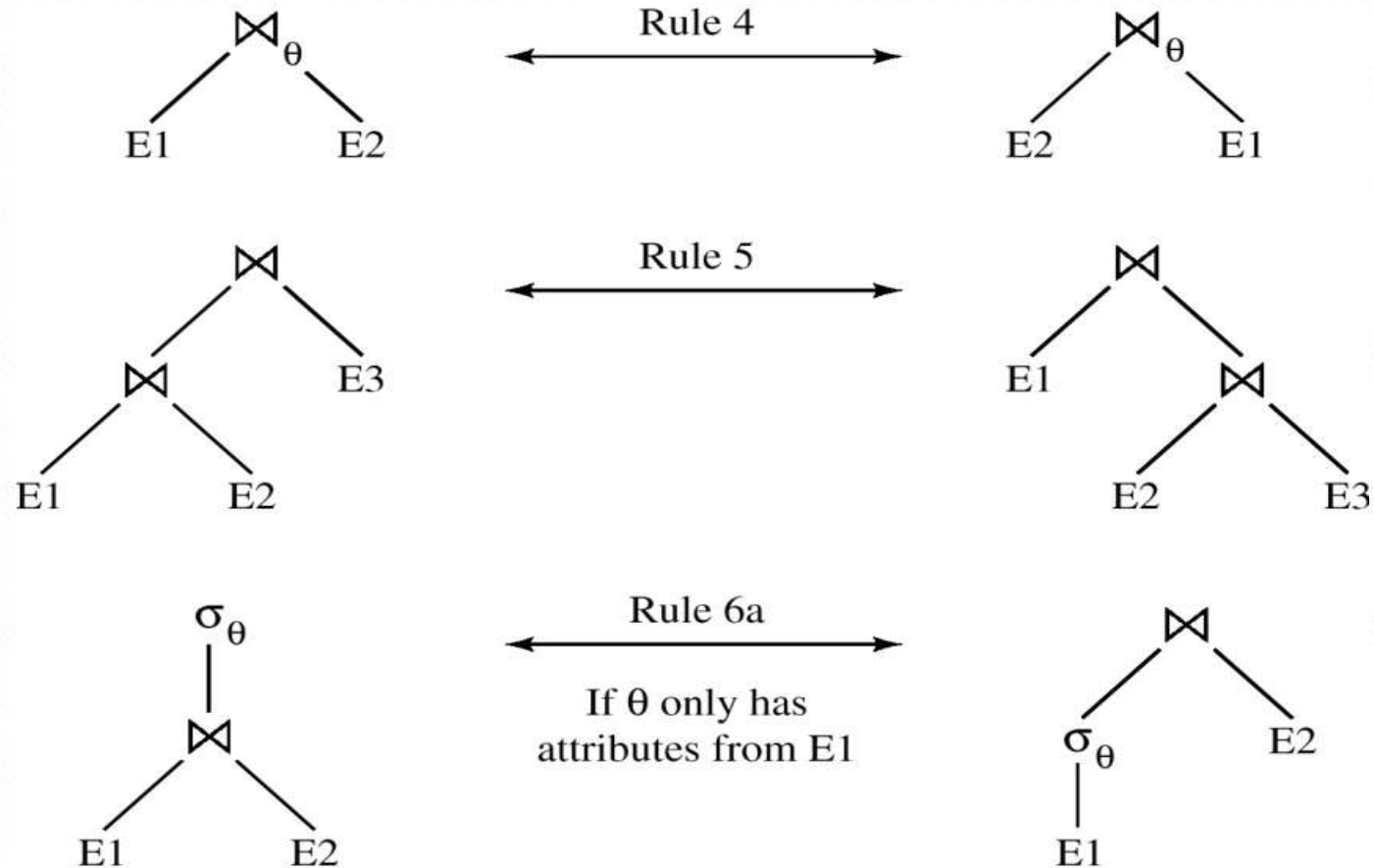


Fig.16.3 Pictorial Depiction of Equivalence Rules

Equivalence Rules

■ **Rule 8.** Projection operation distributes over theta-join

(投影操作对于连接操作的分配率,投影下移)

- a. L_1 and L_2 are the attributes of E_1 and E_2 , respectively.
if join condition θ involves only attributes in $L_1 \cup L_2$, then

$$\Pi_{L_1 \cup L_2} (E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1} (E_1)) \bowtie_{\theta} (\Pi_{L_2} (E_2))$$

E1: instructor (**ID**, name, **dept-name**, salary)

E2: department(**dept-name**, building, **budget**)

只选后续用到的属性!

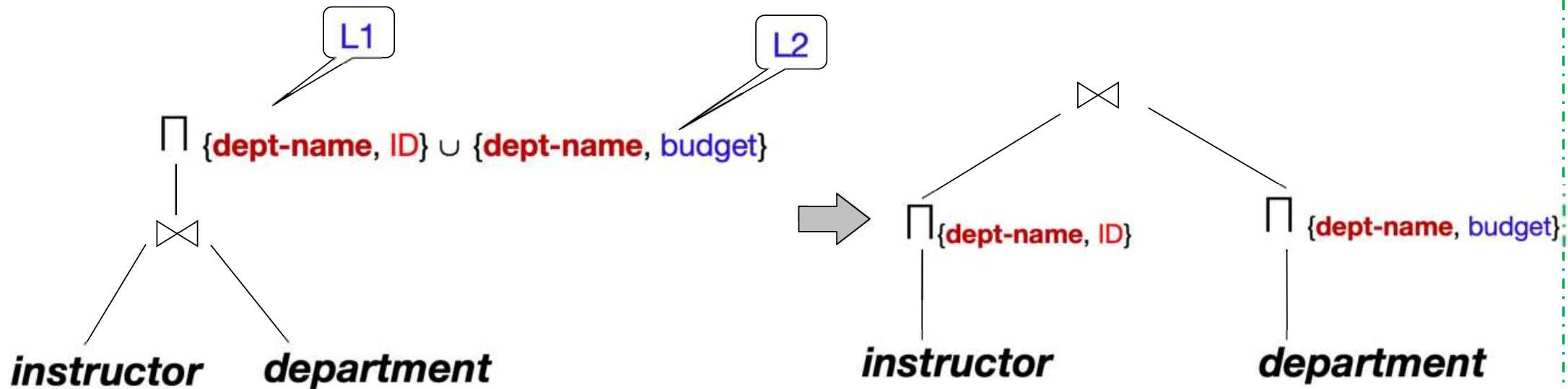


Fig.16.0.3 An example of Rule 8a

Equivalence Rules

- ▣ b. consider a join $E_1 \bowtie_{\theta} E_2$
 - let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively
 - let L_3 be attributes of E_1 that involved in join condition θ , but **not** in $L_1 \cup L_2$,
 - let L_4 be attributes of E_2 that involved in join condition θ , but **not** in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2} (E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2} ((\Pi_{L_1 \cup L_3} (E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4} (E_2)))$$

E1: *instructor* (~~ID~~, **name**, ~~dept-name~~, **salary**)

E2: *department* (~~dept-name~~, **building**, **budget**)

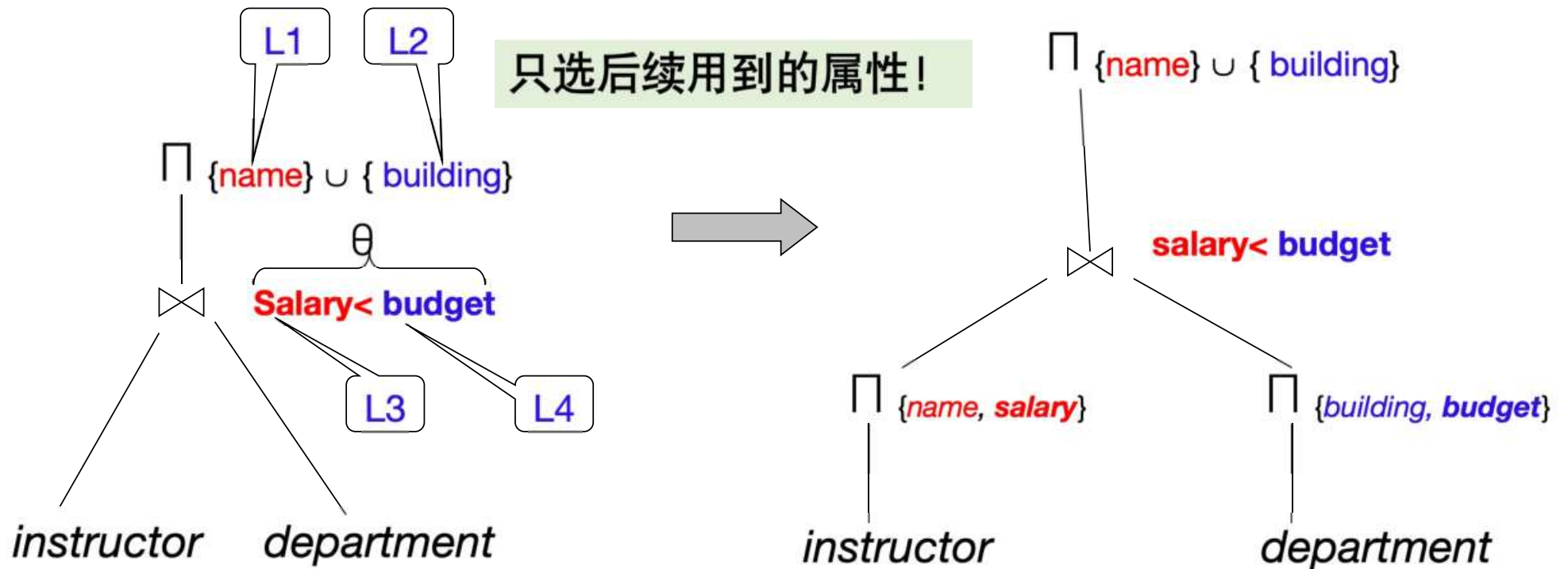


Fig.16.0.4 An example of Rule 8b

Equivalence Rules

- **Rule 9.** The set operations *union* and *intersection* are **commutative**

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

- set difference is not commutative

- **Rule 10.** Set *union* and *intersection* are **associative**

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

Equivalence Rules

- **Rule11.** (选择操作对于集合并、交、差的分配率)

The selection operation distributes over $\cup$, $\cap$ and $-$.

- $\sigma_{\theta} (E_1 - E_2) = \sigma_{\theta} (E_1) - \sigma_{\theta}(E_2)$

- similarly for $\cup$ and $\cap$ in place of $-$

- **Rule12.** (投影操作对于集合并的分配率)

The projection operation distributes over union

- $\pi_L(E_1 \cup E_2) = (\pi_L(E_1)) \cup (\pi_L(E_2))$

Example One: Pushing Selections

- **Example:** Find the *names* of all *instructors* in the department *Music*, along with *titles* of the *courses* that they *teach*

- $\Pi_{name, title} (\sigma_{dept_name = Music} \{ \underline{instructor} \bowtie (teaches \bowtie \Pi_{course_id, title} (course)) \})$

- Transformation using rule 7a.

- $\Pi_{name, title} (\underline{(\sigma_{dept_name = Music} (instructor))} \bowtie (teaches \bowtie \Pi_{course_id, title} (course)))$

- Performing **selection** as early as possible reduces size of relation to be joined.

Example Two: Multiple Transformations

- **Example:** Find **names** of all **instructors** in **Music** department who taught in **2019**, along with **titles** of the **courses** that they taught

- $\Pi_{name, title}(\sigma_{dept_name = Music \wedge year = 2019}$
 $\{instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))\})$ 

- Transformation using join **associative** (Rule 6a):

$$\Pi_{name, title}(\sigma_{dept_name = Music \wedge year = 2019}$$
$$((instructor \bowtie teaches) \bowtie \Pi_{course_id, title}(course)))$$

- Apply **the selections early rule**, resulting in the sub-expression (Rule 7):

$$\sigma_{dept_name = Music}(instructor) \bowtie \sigma_{year = 2019}(teaches)$$

Example Three: Pushing Projections

- **Example:** $\Pi_{name, title}((\sigma_{dept_name = Music}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$

- Computing

$(\sigma_{dept_name = Music}(instructor) \bowtie teaches)$

resulting in a schema:

(ID, name, dept_name, salary, course_id, sec_id, semester, year)

- **Projections** eliminate unneeded attributes from intermediate results:

$\Pi_{name, title}(\Pi_{name, course_id}((\sigma_{dept_name = Music}(instructor) \bowtie teaches)) \bowtie \Pi_{course_id, title}(course))$

- **Projection** as early as possible reduces size of the relation to be joined.

16.3 Join Ordering

- For all relations r_1, r_2 and r_3

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

- If $r_2 \bowtie r_3$ is quite **large** but $r_1 \bowtie r_2$ is **small**, choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we store a **smaller** temporary relation.

- 优化问题，优化目标：产生的中间结果/临时关系最小化

原则：

size小的关系作为join操作中的outer关系

Example

- **Example:** Consider the expression

$$\pi_{name, title}(\sigma_{dept_name = Music}(instructor) \bowtie teaches \bowtie \pi_{course_id, title}(course))$$

- 1. Compute $(teaches \bowtie \pi_{course_id, title}(course))$ first, and join result with

$$\sigma_{dept_name = Music}(instructor)$$

as the result of the first join is likely to be a large relation.

- 2. If **only a small fraction** of instructors are likely to be

- it is better to first compute

$$\sigma_{dept_name = Music}(instructor) \bowtie teaches$$

16.4 Choice of Evaluation Plans: Query optimization

- **Query optimization**

- ▣ **choosing** an evaluation plan with the **lowest** or **lower cost**

- **Generate** an **evaluation plan** for a relational algebra expression with the **lowest** or **lower cost**

- ▣ **1. generating** the **optimized** query **trees**
 - ▣ **2. selecting**
 - **implementation algorithms** for each operations in the tree
 - **pipeline** or **materialization** strategy

Two Approaches for Choice of Evaluation Plans

- Practical query optimizers incorporate elements of two approaches:
 - 1. Cost-based (16.4.1, 16.4.2) 【略】
 - Search all plans and choose the best plan in a cost-based fashion.
 - 2. **heuristic optimization** (16.4.3) 【掌握】
 - Uses **heuristics** to choose a plan

16.4.3 Heuristic Optimization

■ Heuristic optimization

- **heuristics** (or **heuristic rules**) to choose **a better** plan
- **suboptimal** plan with **lower** costs can be obtained

■ Principles: **transforms** query-tree by **heuristics** to reduce cost

- perform **selection** early to **reduce** the number of **tuples**
- perform **projection** early to **reduce** the number of **attributes**
- perform **join** to substitute **Cartesian product** and **selection**
- perform **restrictive selection** and **join** operations before other operations
 - the most **restrictive operations** generate relations with **the smallest size**

//先执行这些操作

Steps in Heuristic Optimization

- Step1: 将**conjunctive selection** 分解为多个单独的**选择操作**,
以使**单个选择操作**尽可能沿查询树下移 (早**选择操作**, **减少中间计算结果**)
- Step2 根据**选择操作**的交换率和分配率, 将查询树每个**选择操作**尽可能移向叶节点,
以便尽早执行**选择操作**
- Step3 根据**连接操作**的结合律和交换率, 重新安排查询树中的叶结点,
使得具有**restrictive selection**特征的叶结点先执行
 - ▣ **restrictive selection**: 执行此操作, **产生的结果关系最小** (所含元组最少)
- Step4 利用以**连接操作**代替相邻的**选择和笛卡尔乘积操作**
- Step5 将查询树上**投影操作**尽可能下移, 早执行**投影操作**, **减少中间计算结果**

Example One

- **Example:** Given
 - **S** (S#, sname, age, sex)
 - C (C#, cname, teacher)
 - **SC** (S#, C#, grade)
- Find **students**, who are **male** and get **grades** more than 90, when they learn some **course**
- Step1. SQL statement
 - SELECT DISTINCT *sname*
FROM **S**, **SC**
WHERE S.s# = SC.s# AND **sex=male** AND **grade > 90**

join condition

selection conditions

Example One (cont.)

■ Step1. Initial Relational algebra expression

□ E1 =

$$\Pi_{\text{sname}}(\sigma_{\text{s.s\#}=\text{sc.s\#} \wedge \text{sex}=\text{M} \wedge \text{grade}>90} (\text{S} \times \text{SC}))$$

□ 或: $\Pi_{\text{sname}}(\sigma_{\text{sex}=\text{M} \wedge \text{grade}>90} (\text{S} \bowtie \text{SC}))$

说明:

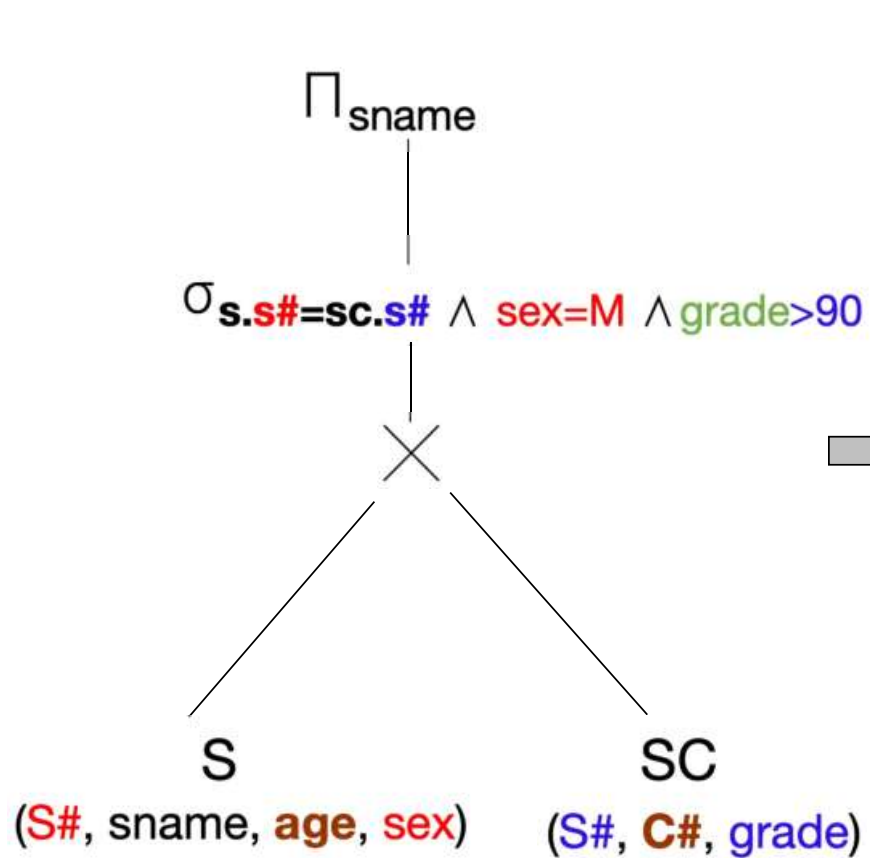
1. 实际查询优化时, 直接用连接替代笛卡尔积操作, 不会使用耗时费力的笛卡尔积
2. 初始关系代数表达式中直接使用连接操作, 不要出现笛卡尔积, 更简洁

■ Step2. Initial query tree

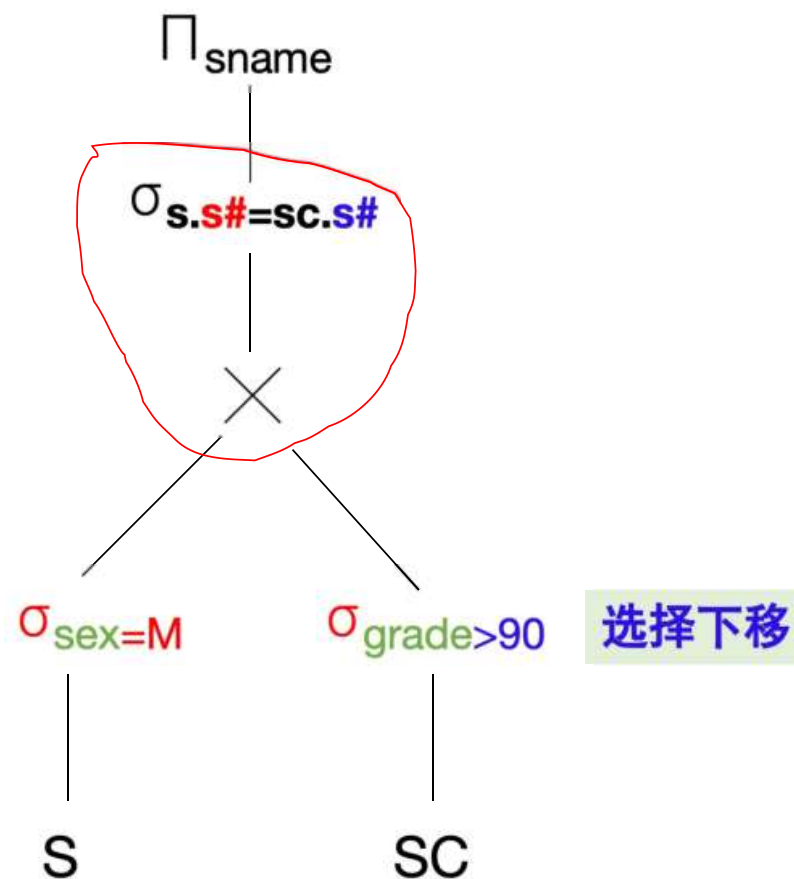
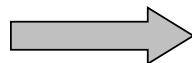
□ Fig. 16.0.5 (a), E1

■ Step3. Distribute σ operations over relation **S** and **SC**

□ Fig. 16.0.5 (b), E2



(a) Initial query tree
E1



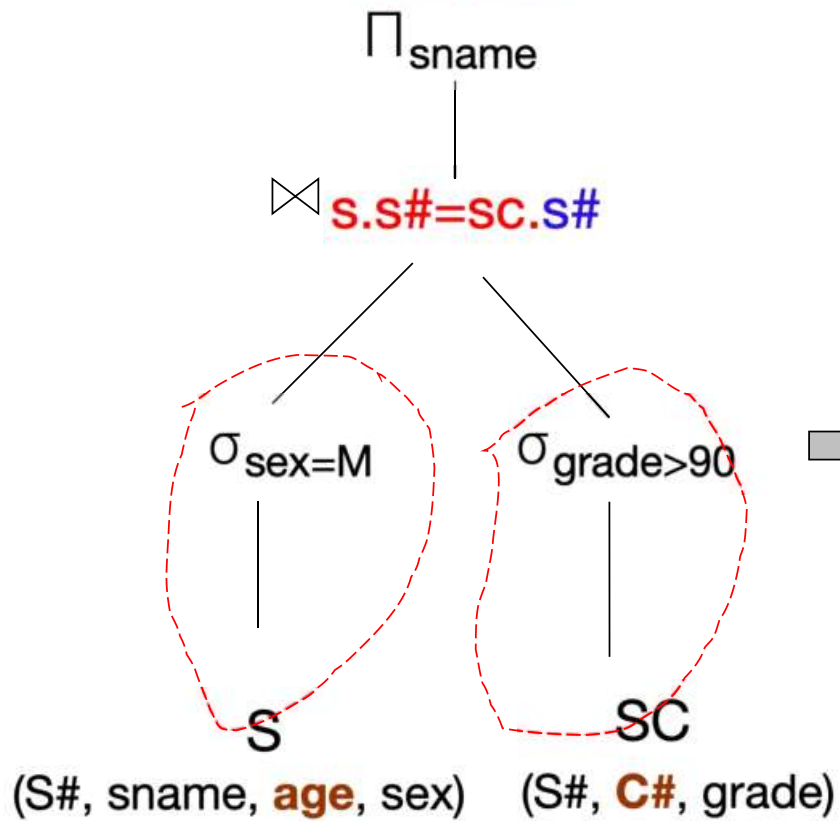
(b) E2

Fig. 16.0.5

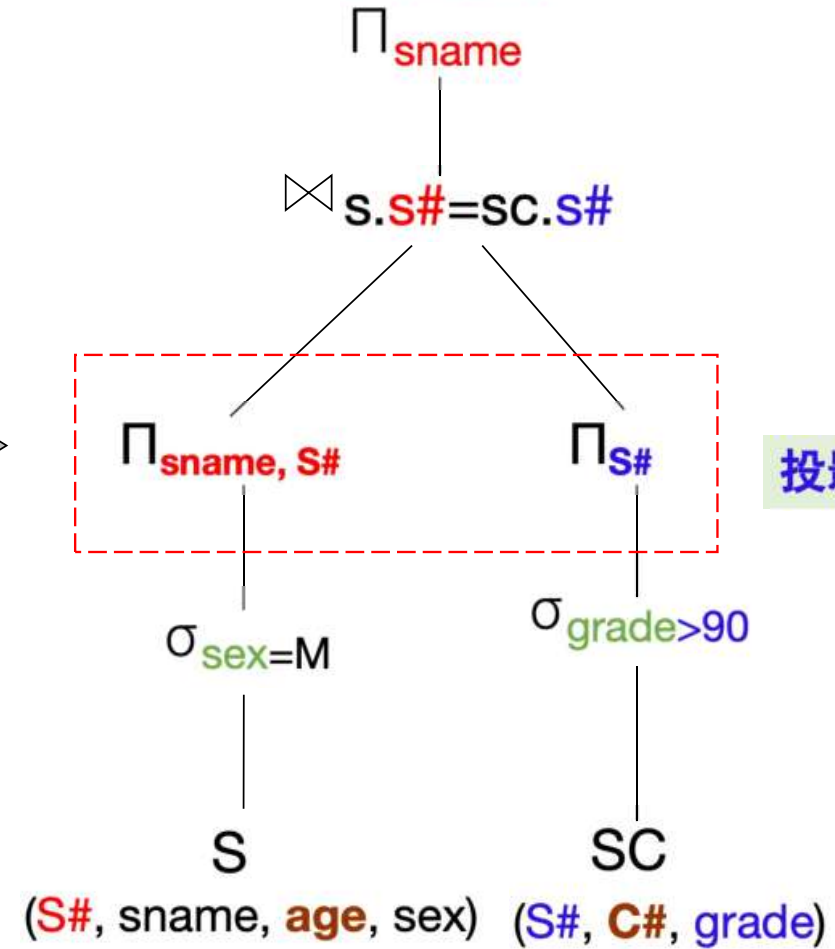
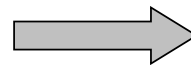
Example One (cont.)

- Step5. Replace **selection** and **Cartesian product** with **natural join**, Rule4.a
 - Fig. 16.0.5 (c), E3
- Step6. Distribute Π operations over relation **S** and **SC**, Rule8.b
 - Fig. 16.0.5 (d), E4
- The final optimized expression E4 equivalent to initial expression E1

$$\square \Pi_{\text{sname}} \{ \Pi_{\text{sname}, S\#} (\sigma_{\text{sex}=\text{M}} (S)) \bowtie \sigma_{\text{grade}>90} (\Pi_{\text{grade}, S\#} (SC)) \}$$



(c) E3



(d) E4

Example Two

- **Example:** Consider the insurance DB

- **Person**(driver-id, name, address)
- **Car**(license, model, year)
- **Accident**(report-number, date, location)
- **Participated**(driver-id, license, report-number, damage-amount)

- **Problems**

- Find out the **driver name**, **license**, **report-number** of the **accidents** which happened in **Beijing** and before **May 3, 2024**
- Give the initial query tree for this query
- Construct an optimized **relational algebra expression**

Example Two (cont.)

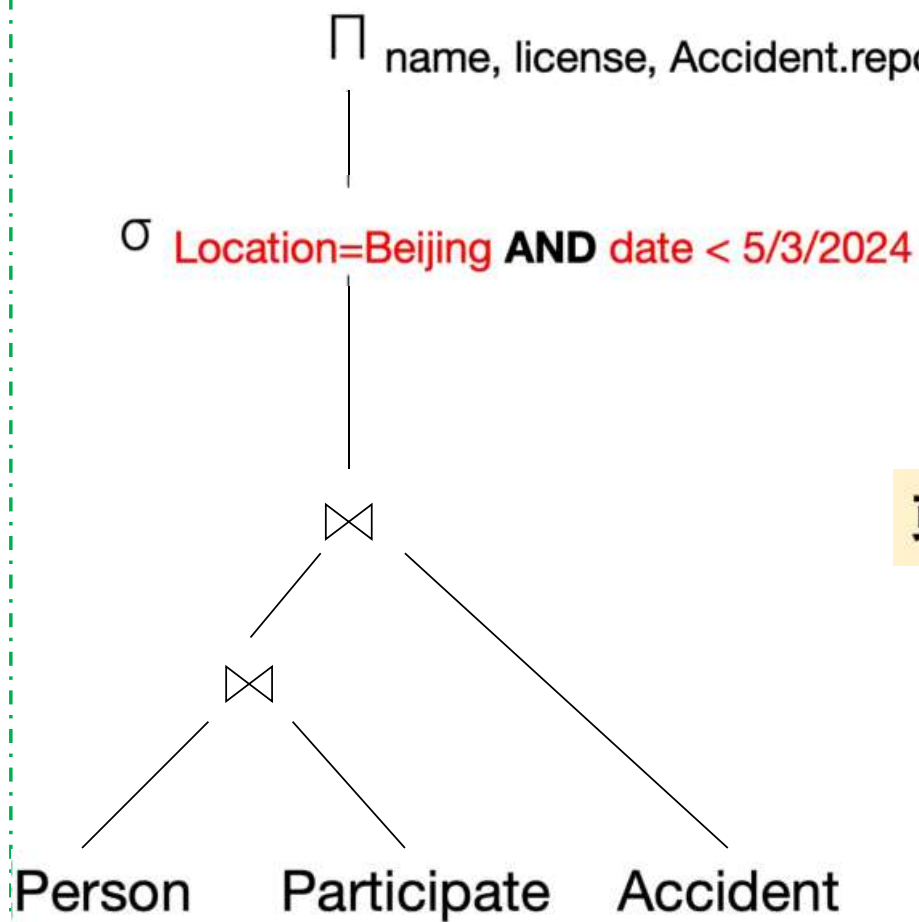
■ Step1. SQL statement

- ▣ **Select** name, license, Accident.report-number
From *Person, Participate, Accident*
Where Person.driver_id = Participate.driver_id 注意：不需要关系Car
 AND Accident.report_number = Participate.report_number
 AND Location=Beijing AND date < 5/3/2022

■ Step2. Initial expression

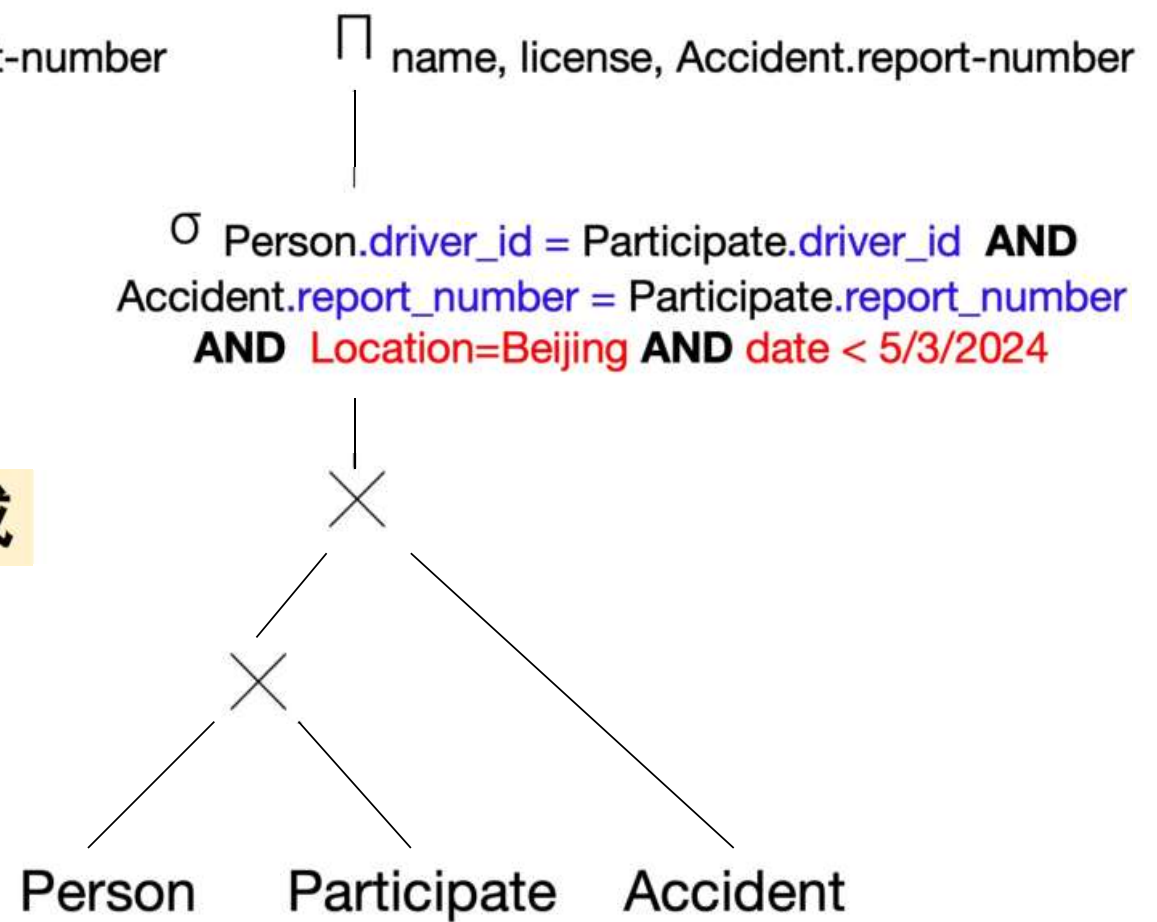
Π name, license, Accident.report-number

$(\sigma_{\text{Person.driver_id} = \text{Participate.driver_id} \text{ AND } \text{Accident.report_number} = \text{Participate.report_number} \text{ AND } \text{Location}=\text{Beijing} \text{ AND } \text{date} < 5/3/2022}$
 $(\text{Person} \times \text{Participate} \times \text{Accident})$

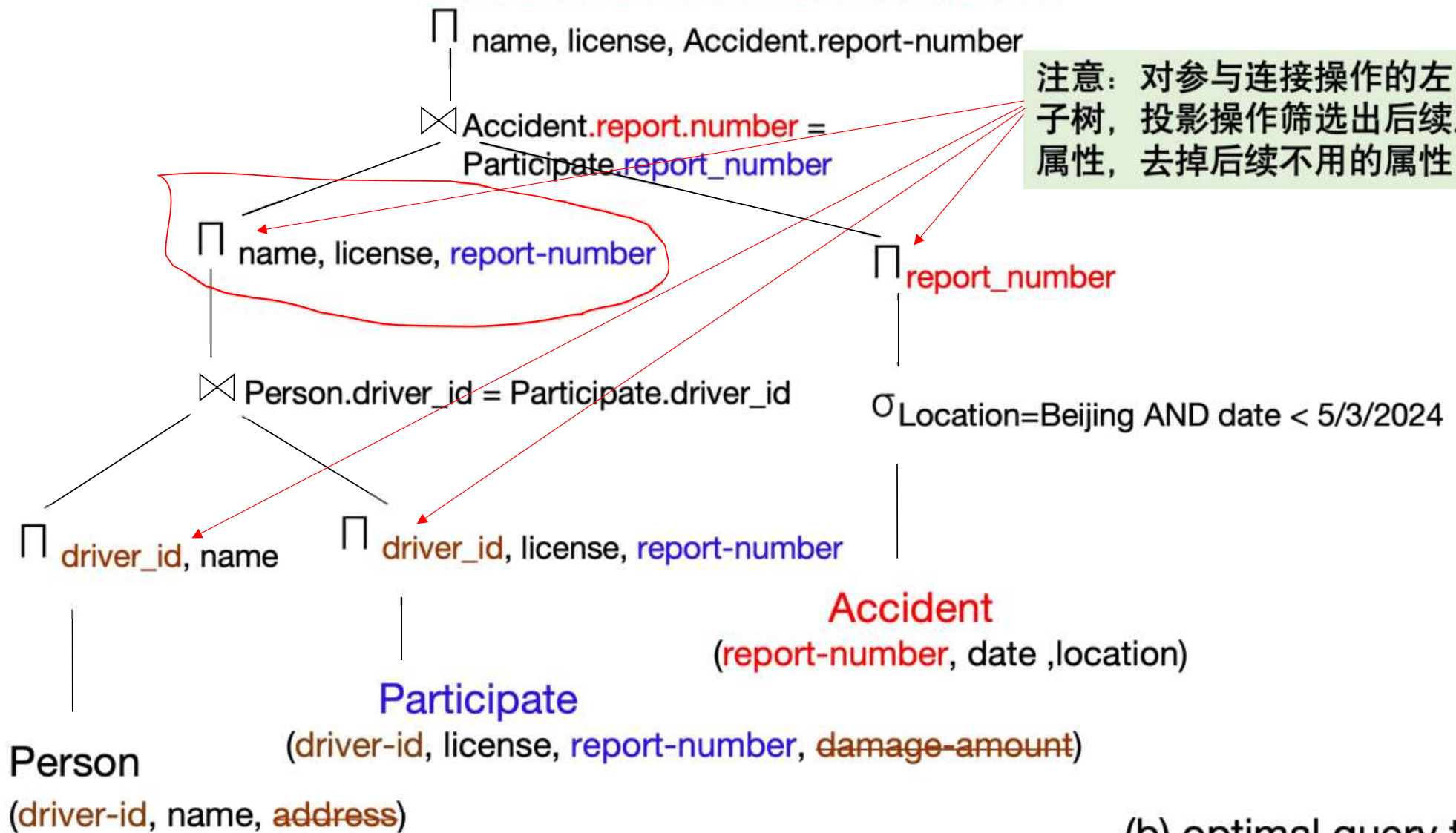


(a) initial query tree

或



(a) initial query tree



(b) optimal query tree

Example Three

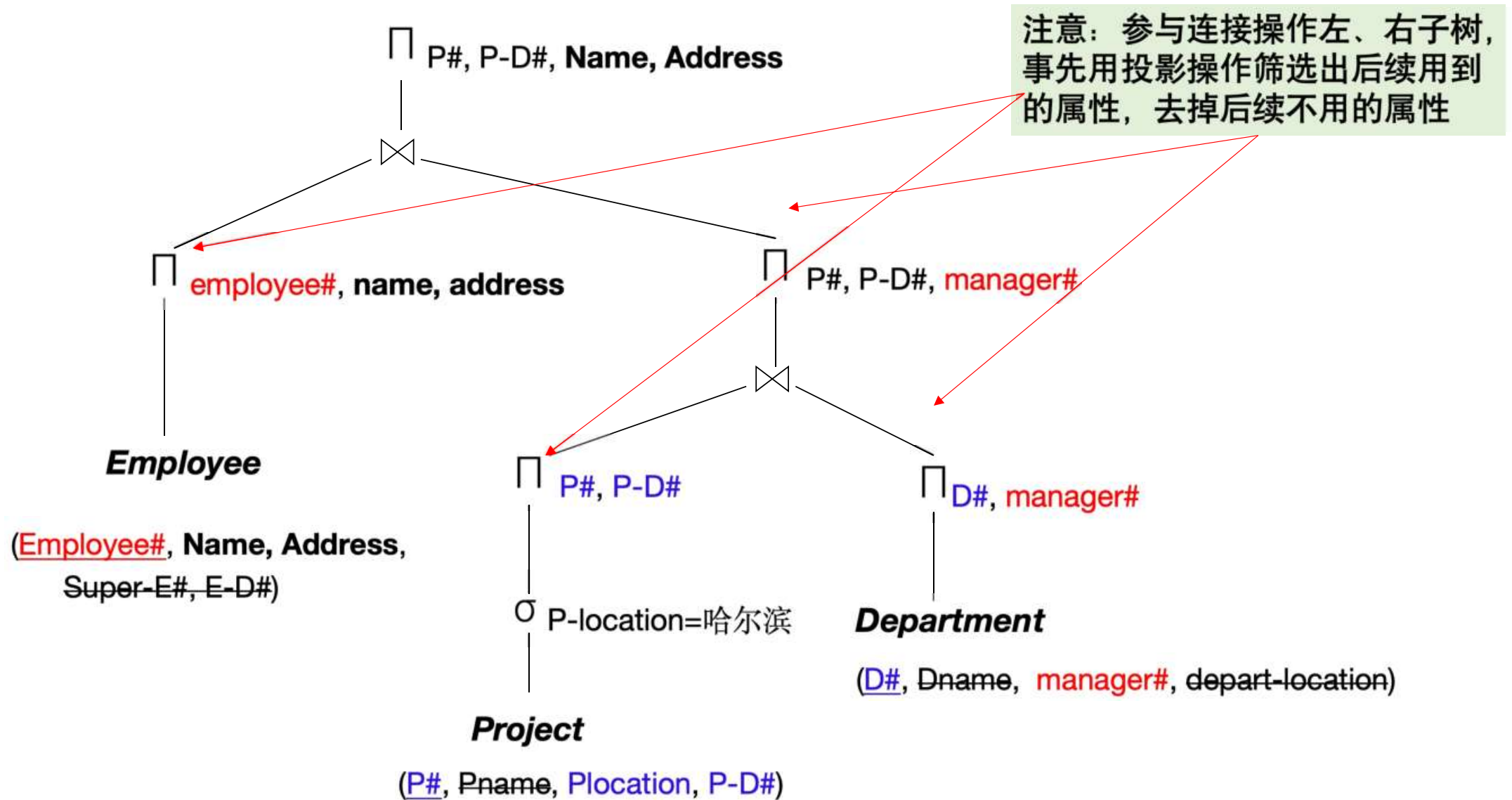
- **Example:** 给定如下关系数据库

Employee (Employee#, Name, Address, Super-E#, E-D#)

Department (D#, Dname, manager#, depart-location)

Project (P#, Pname, Plocation, P-D#)

- 查询：对于哈尔滨的工程项目，列出其工程项目号P#，工程所属部门号P-D#，该部门领导的名字Name和地址Address
- 要求：写出查询SQL语句；转换为关系代数表达式并利用启发式方法进行查询优化；
- Select P#, P-D#, Name, Address
From Project, Department, Employee
Where Plocation =哈尔滨 AND P-D# = D# AND manager# = Employee#



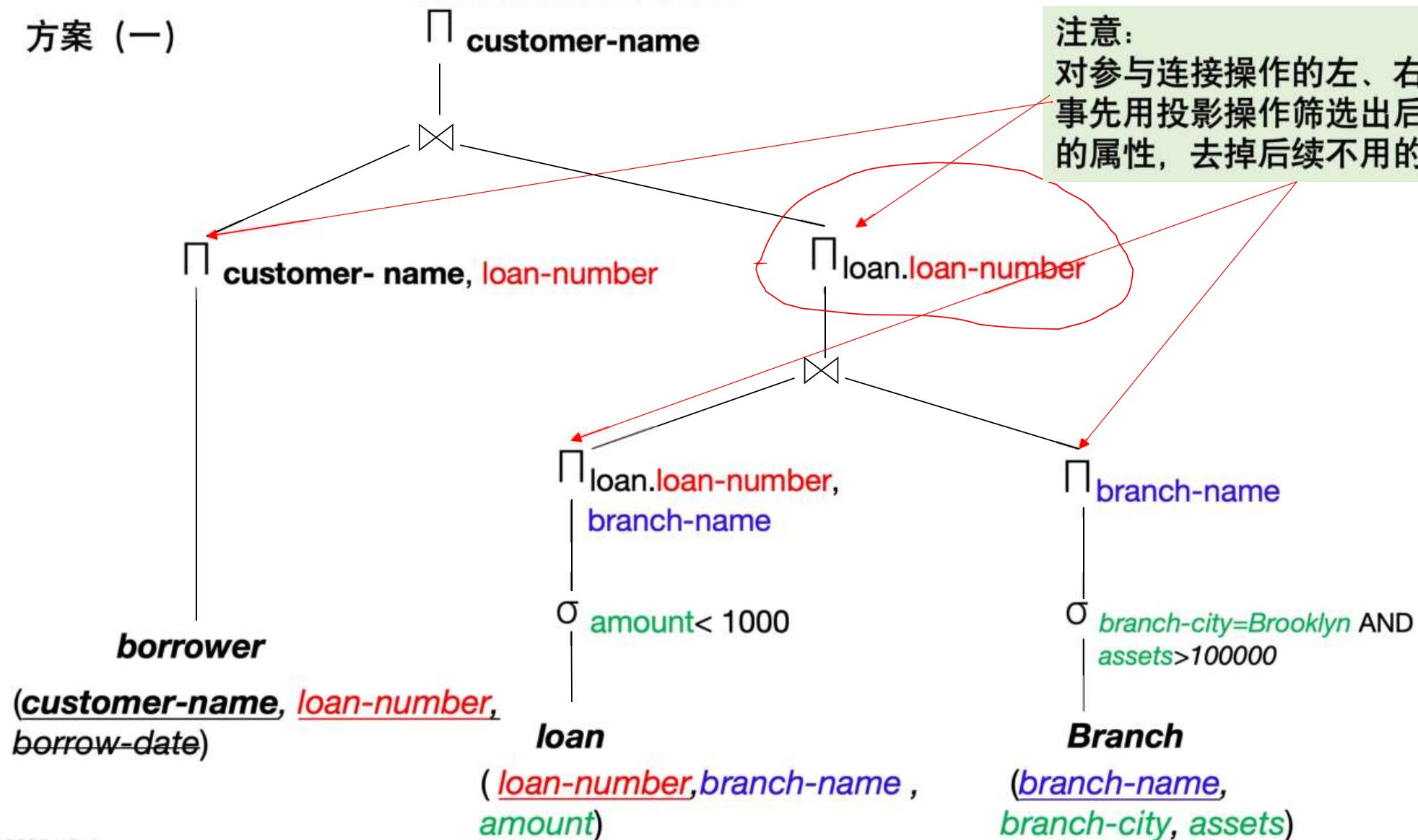
Example Four

- **Example:** Consider relations in banking enterprise database
 - **branch** (branch-name, branch-city, assets),
 - **loan** (loan-number, branch-name , amount)
 - **borrower**(customer-name, loan-number, borrow-date)
 - **customer** (customer-name, customer-street, customer-city)
 - **account** (account-number, branch-name, balance)
 - **depositor** (customer-name, account-number, deposit-date)
- Find **names** of **customers** who have an **loan** at **branch** located in **Brooklyn** and have **assets** more than \$100,000, requiring **loan-amount** is less than \$1000
 - give an SQL statement
 - given **an optimized query tree**

❑ select customer-name
from *borrower*, *loan*, *branch*
where loan.loan-number=borrower.loan-number
and branch.branch-name=loan.branch-name
and branch-city=Brooklyn
and assets>100000
and amount<1000

或者: select customer-name
from borrower *natural-join* loan *natural-join* branch
where branch-city=Brooklyn
and assets>100000
and amount<1000

方案 (一)



方案 (二)

哈希匹配



customer-name

表扫描



customername, accountnumber

depositor
(*customername*,
accountnumber,
deposit-date)

堆文件

嵌套循环inner join



account.accountnumber

聚集索引扫描



account.accountnumber,
branchname



σ balance < 1000

account
(*accountnumber*,
branchname, *balance*)

B+树

聚集索引查找



branchname



σ branchcity='Brooklyn'
AND assets > 100000

branch
(*branchname*,
branchcity, *assets*)

B+树

基于索引

Chapter 16 Query Optimization

